



Learning for Image Matching and Structure from-Motion

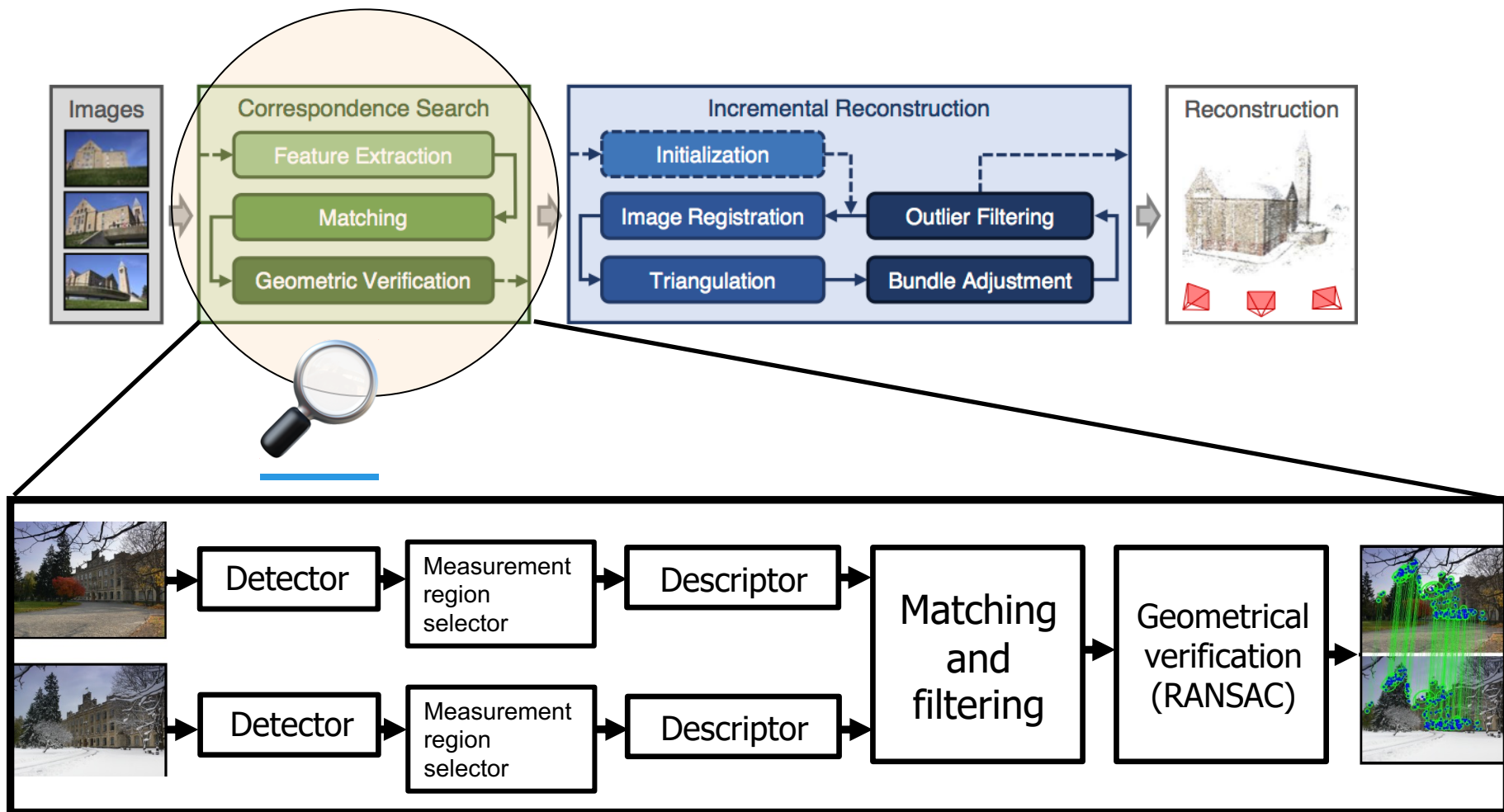
Dmytro Mishkin

Visual Recognition Group, Faculty of Electronical Engineering,
Czech Technical University in Prague

Includes slides by:

- Daniel DeTone, Paul-Eduard Sarlin, MagicLeap
- Jerome Revaud, Naver Labs Europe

Image Matching Pipeline Recap



Why bother with learning for SfM?

Main motivations (from most important) to use learned features:

- Being able to reconstruct things, you could not before
 - Sparse views (just 5-10 images/scene)
 - Day/night, other domains
- To be faster
- To be more precise

RoMA

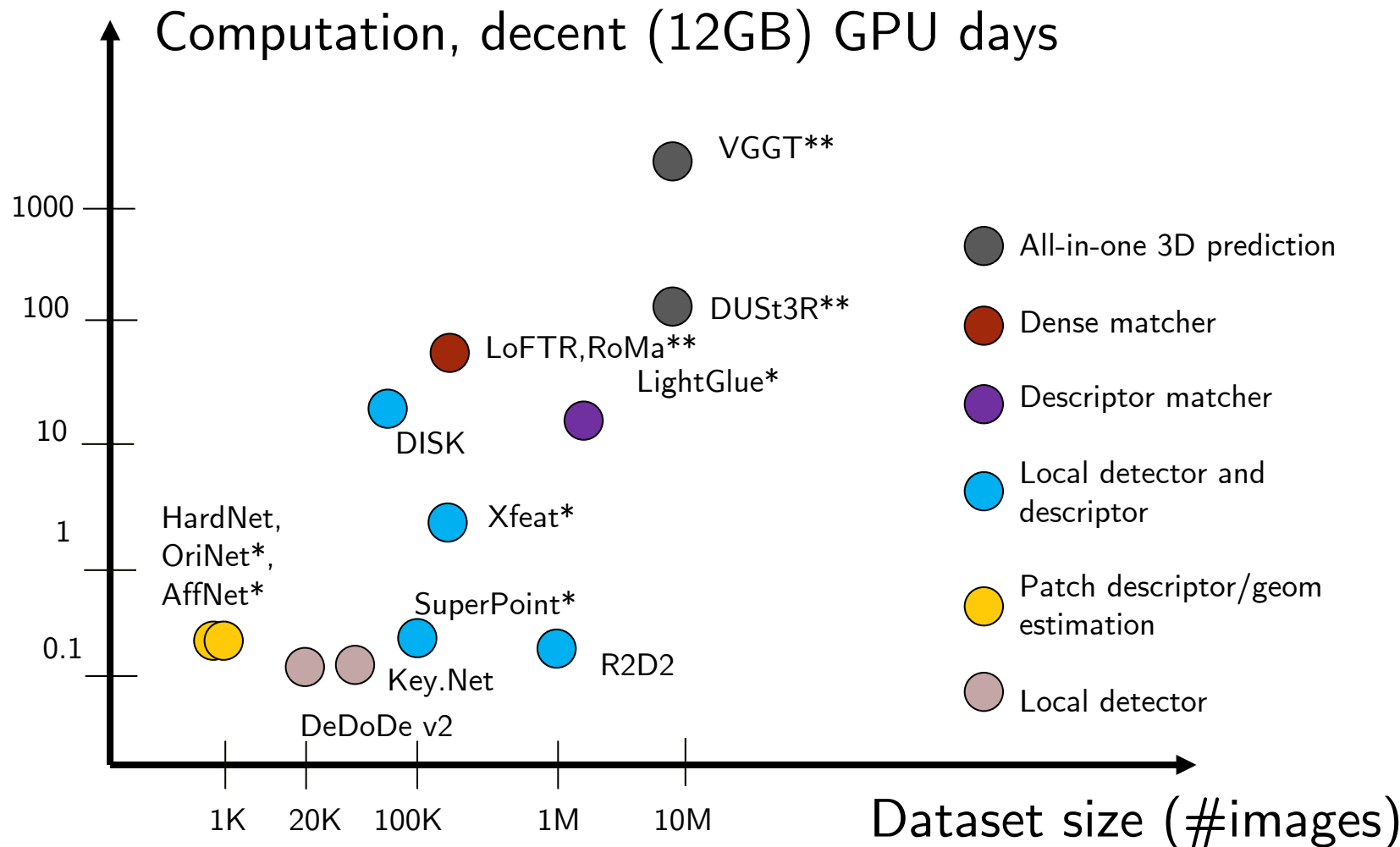


Dust3r



DUST3R: Geometric 3D Vision Made Easy

Learning the WBS pipeline components (or all)



* – significant data augmentation used

** – starting from pretrained model (CroCov2/DINOv2 ViT)

Number of images is a bit wrong metric here:

3 Components of Any Deep Learning Model

■ Data

- Usually the hardest to get/process for image matching
- We need clever augmentations/synthetic data

■ Architecture

- Usually the simplest – just use pick CNN (detector/descriptor) and/or Transformer (matcher)

■ Loss

- Can be easy or hard depending on the task

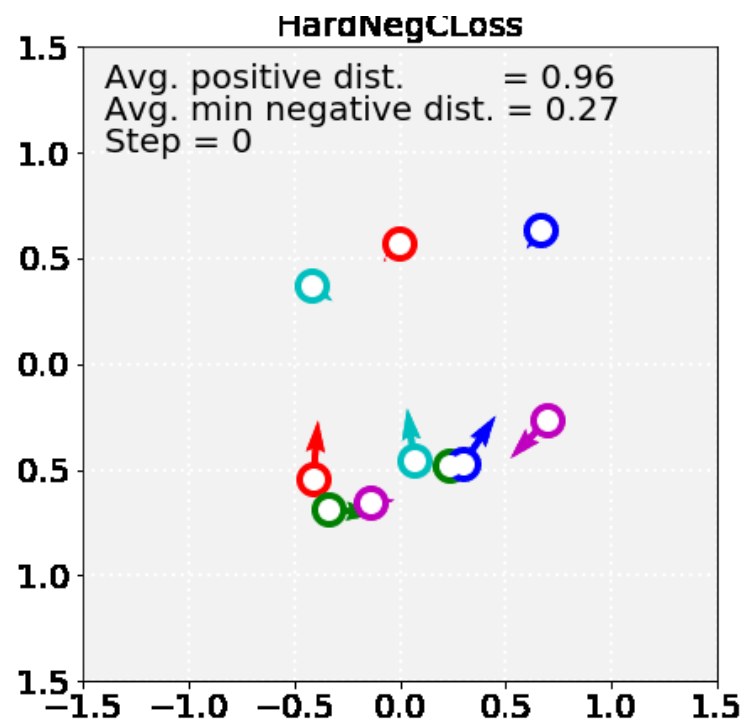
Local descriptor learning

Loss:

- Use any good stateless metric learning loss (contrastive, triplet, AP-based) with hard-negative mining.

$$L_T = \frac{1}{n} \sum_{i=1}^n \max(0, m + d_{\text{pos}_i} - d_{\text{neg}_i})$$

$$L_{AP} = \frac{1}{n} \sum_{i=1}^n 1 - AP_n,$$



Patch training data: Brown dataset

3 sets, 400k patches
each:

- Liberty (shown)
- Notredame
- Yosemite

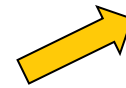
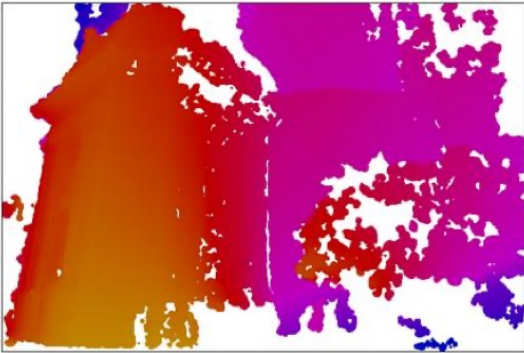
Size: 64x64, grayscale.
Obtained from SfM
model,
3D point $\rightarrow$ DoG
keypoints





1. Detect local features in the reference image
2. Reproject them via depth map to neighboring (“target”) images
 - Check visibility. If not visible, skip feature.
3. Find local features in the target image, such that:
 - position, orientation and scale are in a specific range w.r.t the reprojected point.

R2D2: Optical flow and augmentation supervision



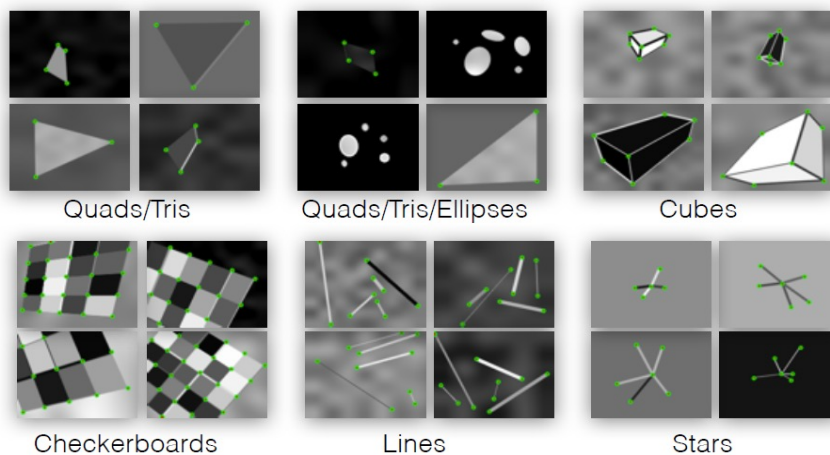
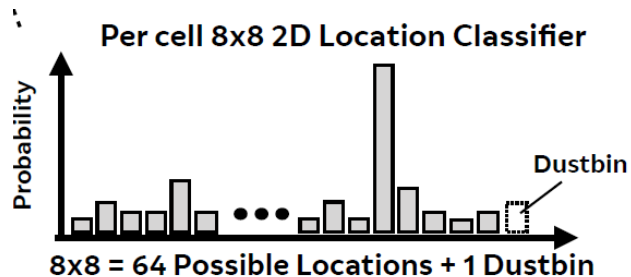
1. Get high-quality optical flow
2. Reproject points via optical flow to neighboring (“target”) images
 - If not confident, skip feature.

Local detector learning

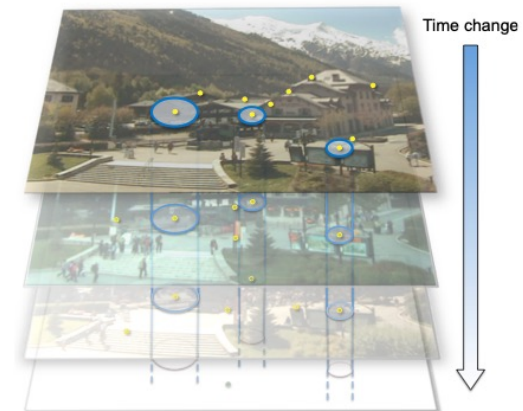
Local detector learning: supervised (when you know what to learn)

Supervised keypoint detector is similar to segmentor

- SuperPoint: you want to learn corners as segmentation
- DeDoDe, Tilde: you want to learn DoG features, but more robust

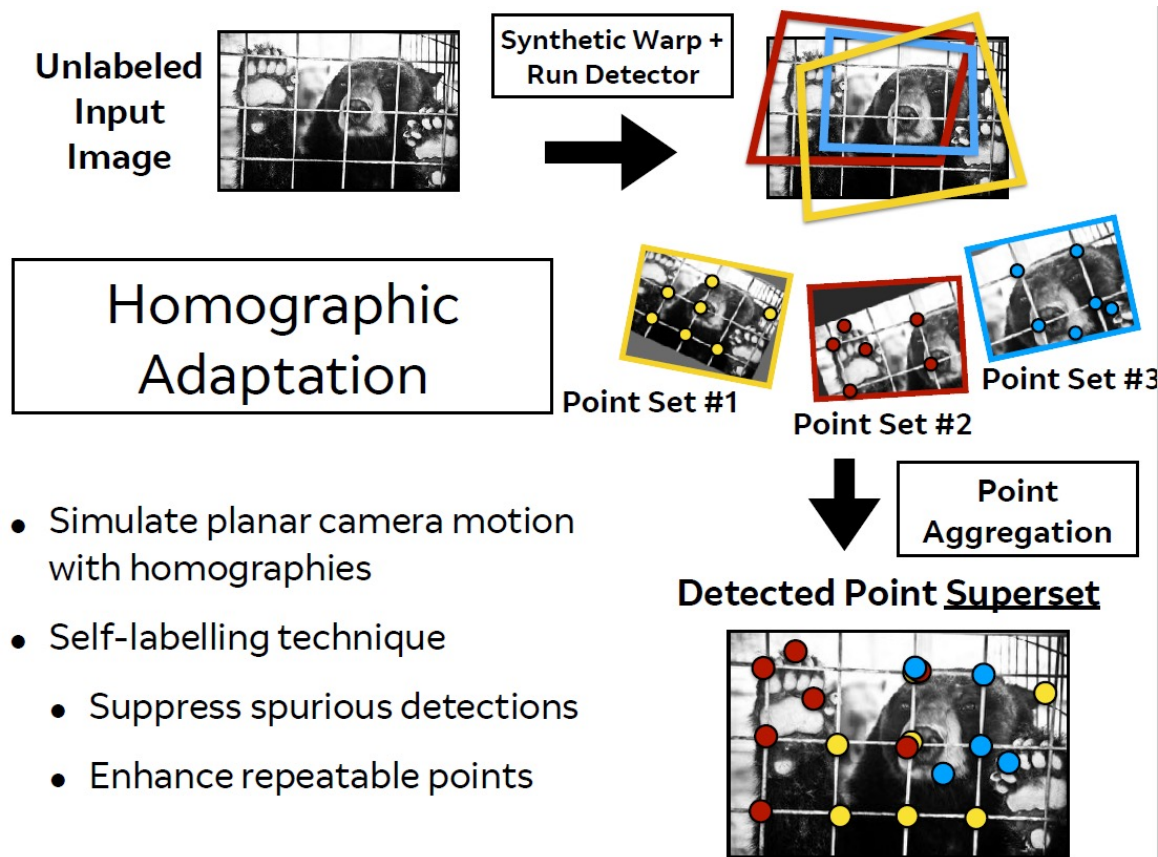


Cross entropy for pixel classification (keypoint or not)



(a) Stack of training images

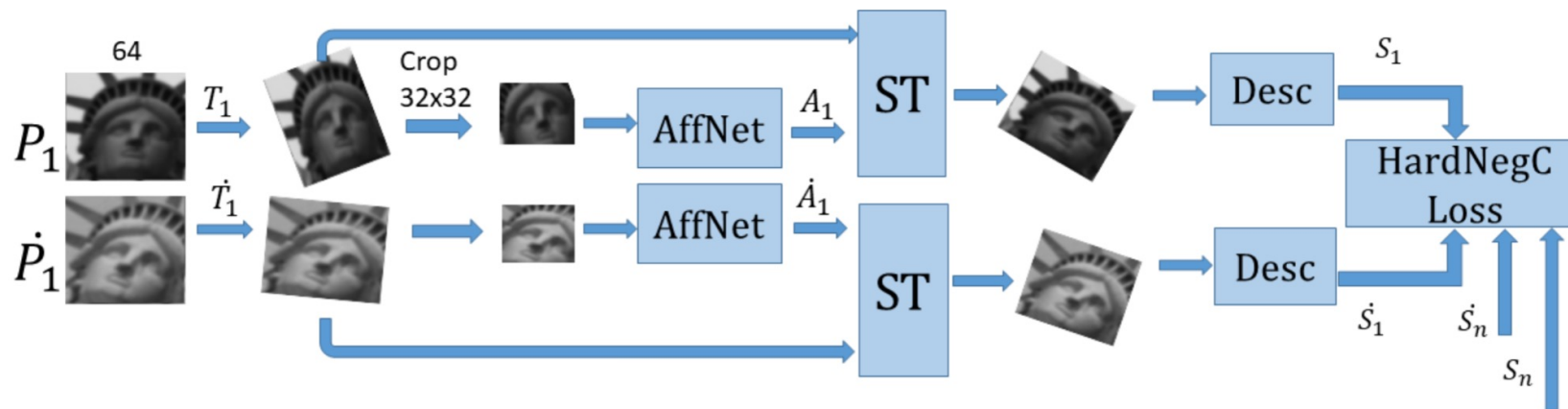
- Synthetic (SuperPoint), static webcam (TILDE), SfM Tracks (DeDoDe)
- Augmentation: homography and color/blur



Local detector discovery: (when you do not know what to learn)

If you know, what result you want, add it to the data and loss function. AffNet/OriNet idea:

- Distorted patches after normalization should look the same



Possible losses:

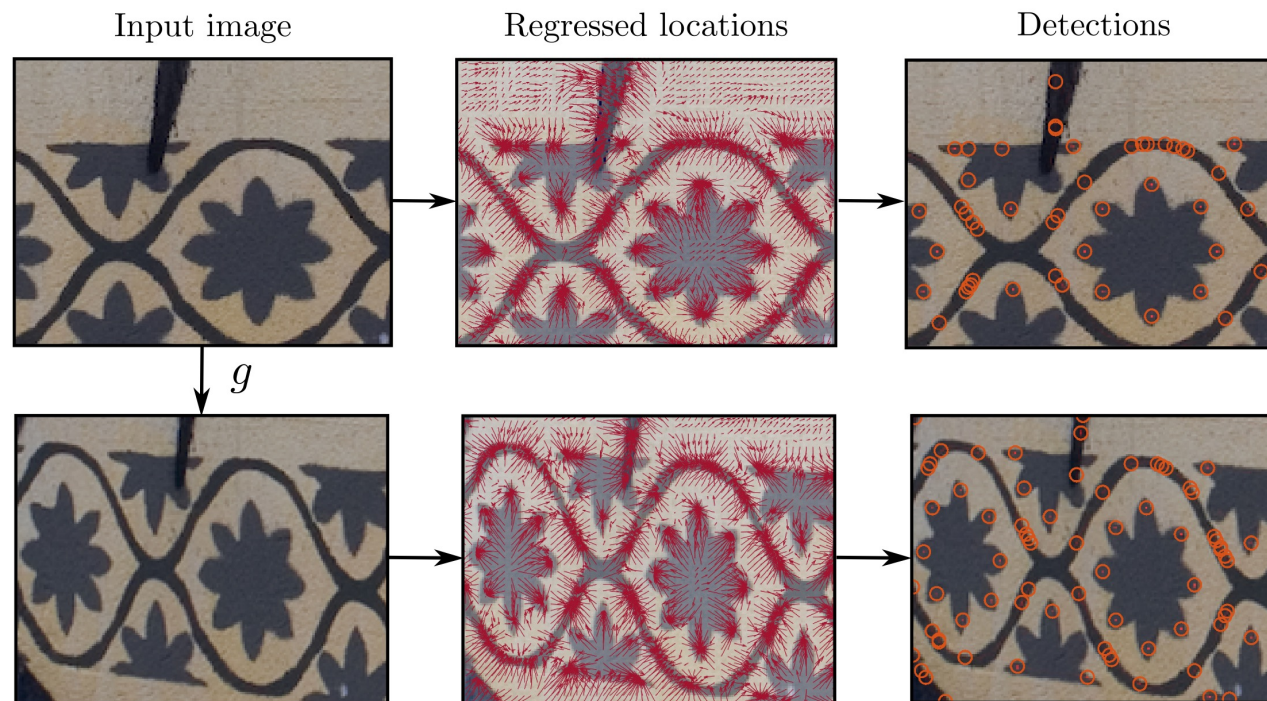
- Patch similarity (pixel level)
- L2 on residual transformation.

Discovering detector: harder

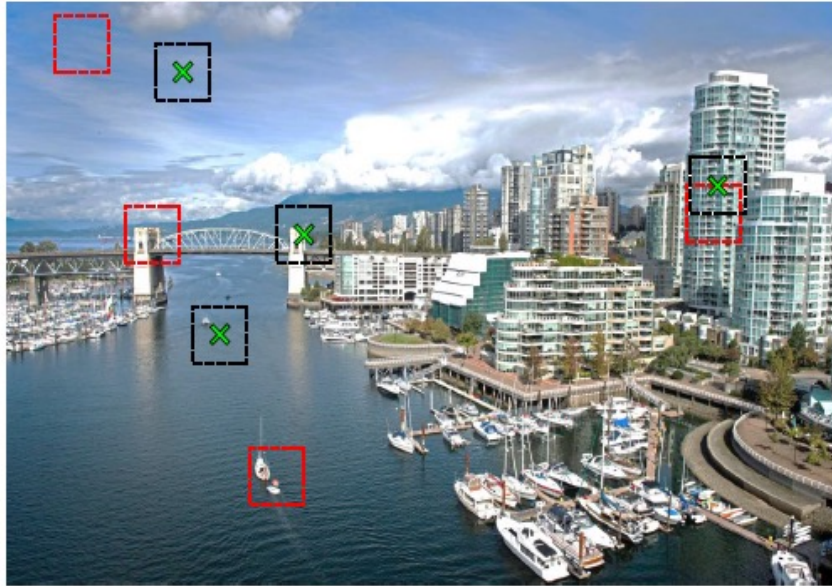
Possible ways:

- Regression-base covariance constraint – DetNet.
 ψ – model, $\mathbf{x}$ – patch, T – transformation

$$\min_{\psi} \frac{1}{n} \sum_{i=1}^n \|\psi(T_i \mathbf{x}_i) - \psi(\mathbf{x}_i) - T_i\|^2$$



Hard to do
non-maxima
suppression
and
thresholding



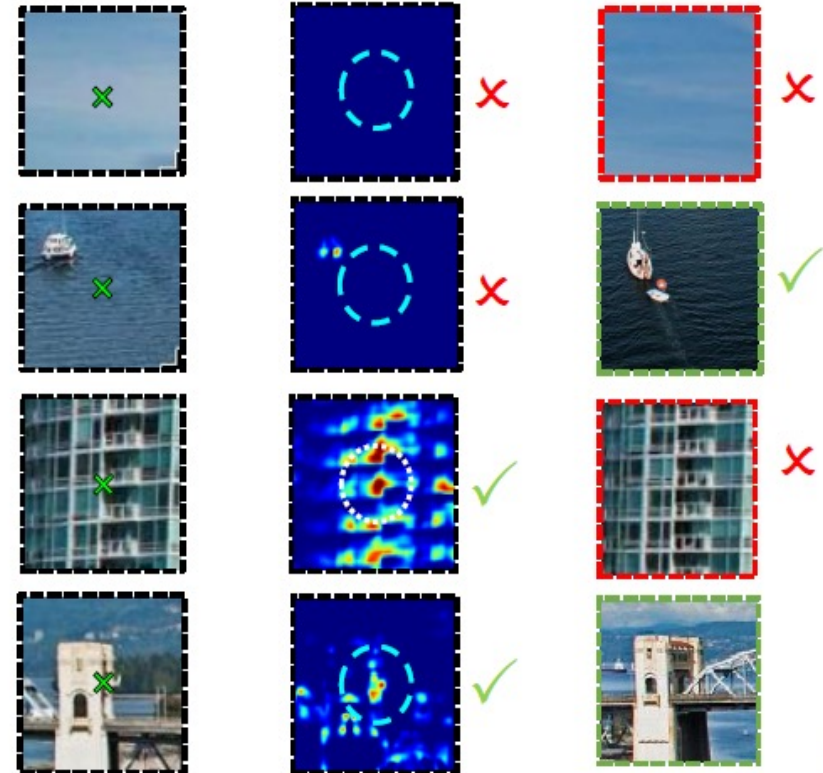
Failure causes:

- The keypoint detector only focuses on *repeatable* locations
- But repeatable locations are not necessarily *reliable* for matching.

© 2019 NAVER LABS. All rights reserved.

What is a good keypoint?

Repeatable? Reliable?
== "matchable"



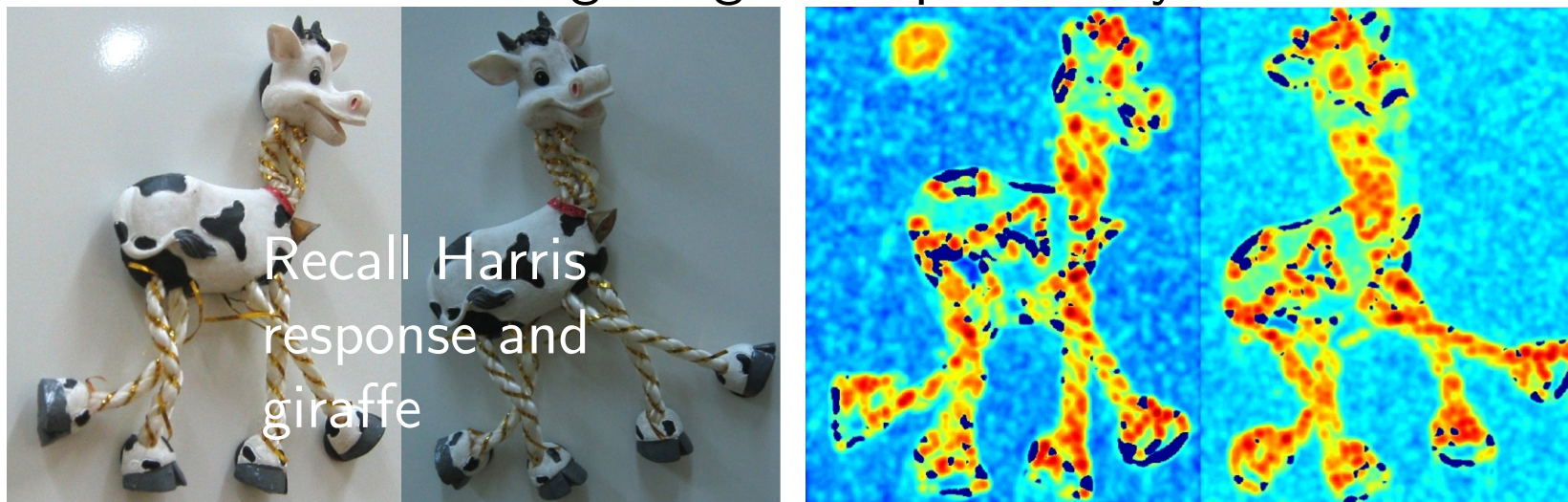
28

Revaud et.al., R2D2: Reliable and Repeatable Detector and Descriptor, NeurIPS 2019

Slide source: [https://neurips.cc/media/Slides/nips/2019/westballc\(12-15-50\)-12-16-25-15782-r2d2_reliable_.pdf](https://neurips.cc/media/Slides/nips/2019/westballc(12-15-50)-12-16-25-15782-r2d2_reliable_.pdf)

Heatmap-base covariance constraint – R2D2:

detector response value should be the same in the corresponding locations of the matching images – repeatability.



$$\mathcal{L}_{cosim}(I, I', U) = 1 - \frac{1}{|\mathcal{P}|} \sum_{p \in \mathcal{P}} cosim(S[p], S'_U[p])$$

U – GT correspondences, S – detector response, P – patch

cosim – cosine similarity $cosim(x, y) = \frac{x^\top y}{\|x\|_2 \|y\|_2}$

- Idea: detector response value should be the same in the corresponding locations of the matching images

$$\mathcal{L}_{cosim}(I, I', U) = 1 - \frac{1}{|\mathcal{P}|} \sum_{p \in \mathcal{P}} cosim(\mathbf{S}[p], \mathbf{S}'_U[p])$$

- Deep networks are lazy. The easiest way to satisfy the condition is to return the constant response everywhere
 - Solution: require response to be “peaky”:

$$\mathcal{L}_{peaky}(I) = 1 - \frac{1}{|\mathcal{P}|} \sum_{p \in \mathcal{P}} \left(\max_{(i,j) \in p} \mathbf{S}_{ij} - \text{mean}_{(i,j) \in p} \mathbf{S}_{ij} \right)$$

Total R2D2 detector loss

U – GT correspondences

S – detector response

P – patch

$$\begin{aligned}\mathcal{L}_{rep}(I, I', U) &= \mathcal{L}_{cosim}(I, I', U) \\ &+ \lambda (\mathcal{L}_{peaky}(I) + \mathcal{L}_{peaky}(I'))\end{aligned}$$

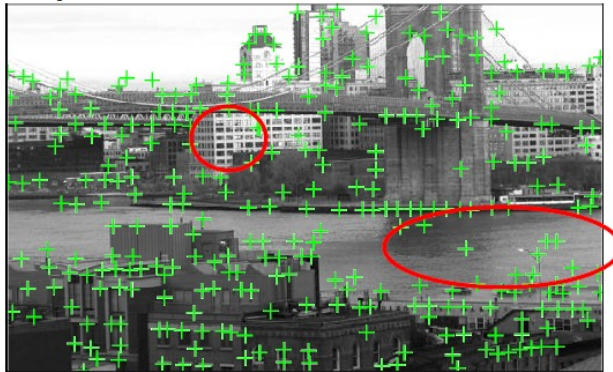
$$\mathcal{L}_{cosim}(I, I', U) = 1 - \frac{1}{|\mathcal{P}|} \sum_{p \in \mathcal{P}} cosim(S[p], S'_U[p])$$

$$\mathcal{L}_{peaky}(I) = 1 - \frac{1}{|\mathcal{P}|} \sum_{p \in \mathcal{P}} \left(\max_{(i,j) \in p} S_{ij} - \text{mean}_{(i,j) \in p} S_{ij} \right)$$

- Idea: predict a descriptor loss (matchability). Combine it with detector score.

Reliability maps

Image



Predicted
reliability



© 2019 NAVER LABS. All rights reserved.

1
6

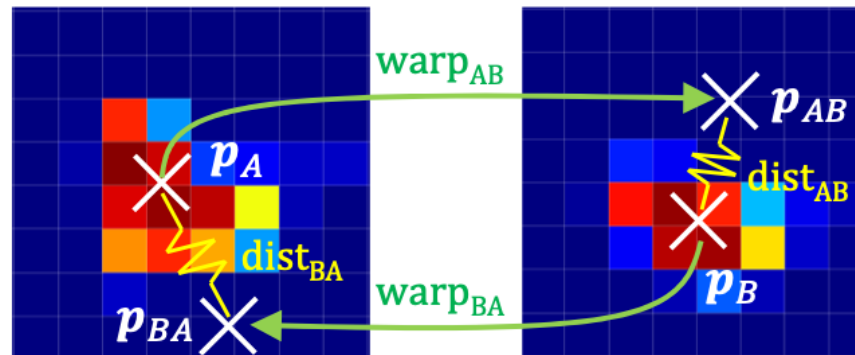
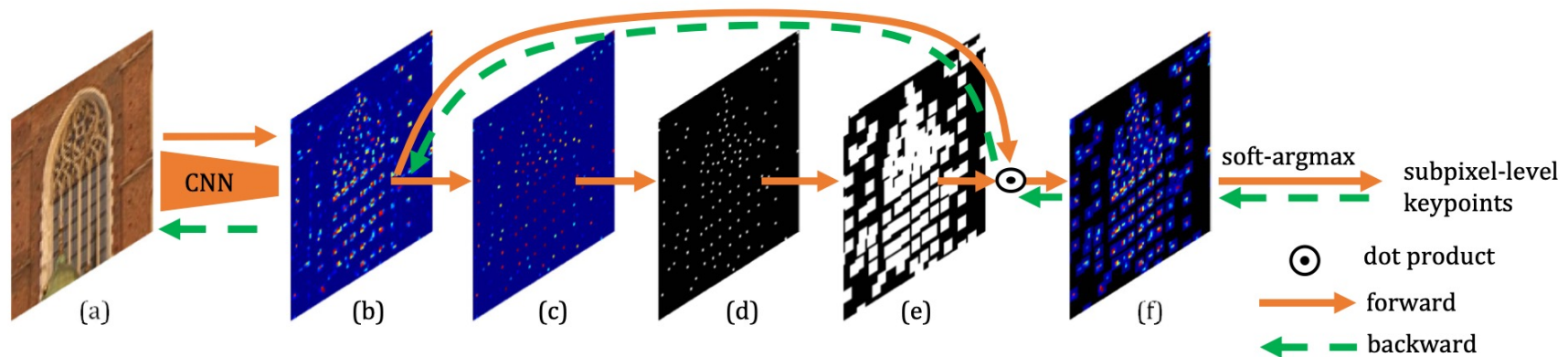
Revaud et.al., R2D2: Reliable and Repeatable Detector and Descriptor, NeurIPS 2019

Slide source: [https://neurips.cc/media/Slides/nips/2019/westballc\(12-15-50\)-12-16-25-15782-r2d2_reliable_.pdf](https://neurips.cc/media/Slides/nips/2019/westballc(12-15-50)-12-16-25-15782-r2d2_reliable_.pdf)

ALIKE(D): R2D2 improved

ALIKE: iteration over R2D2.

- Minimize reprojection loss between scoremap argmax augmentation warps



- DISK, S-TREK, DaD – use Reinforcement Learning to find good keypoints.
 - Good keypoints are those, which you can sample from both images.
 - Reward is proportional to the to inverse of reprojection error (S-TREK, DaD) or +1/-0.25 for DISK

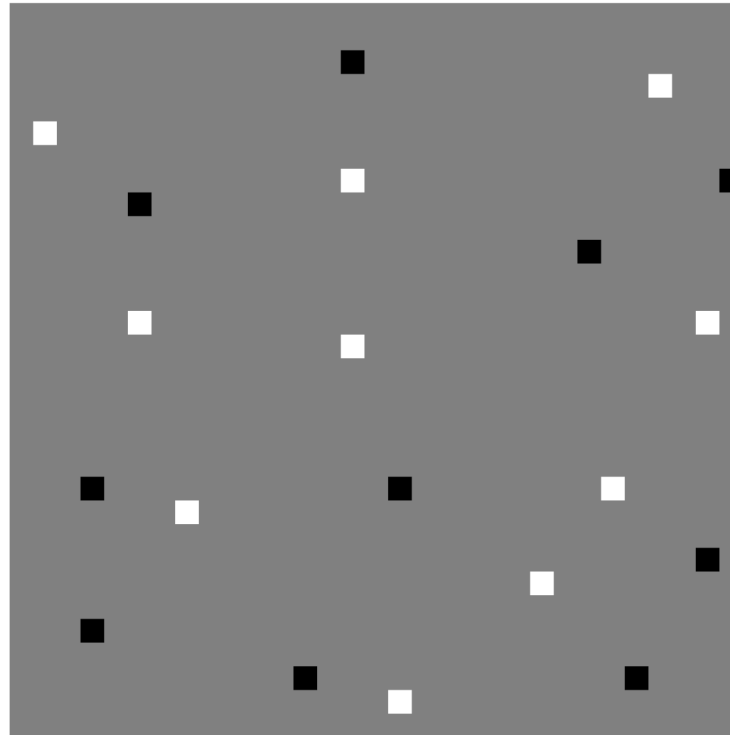
$$r_{k_i^0} = \begin{cases} d_{\max} - d & \text{if } d \leq d_{\max} \\ r_n & \text{otherwise} \end{cases}$$

[DaD: Distilled Reinforcement Learning for Diverse Keypoint Detection](#), 2025

[DISK: Learning local features with policy gradient](#), 2020

[S-TREK: Sequential Translation and Rotation Equivariant Keypoints for local feature extraction](#), 2023

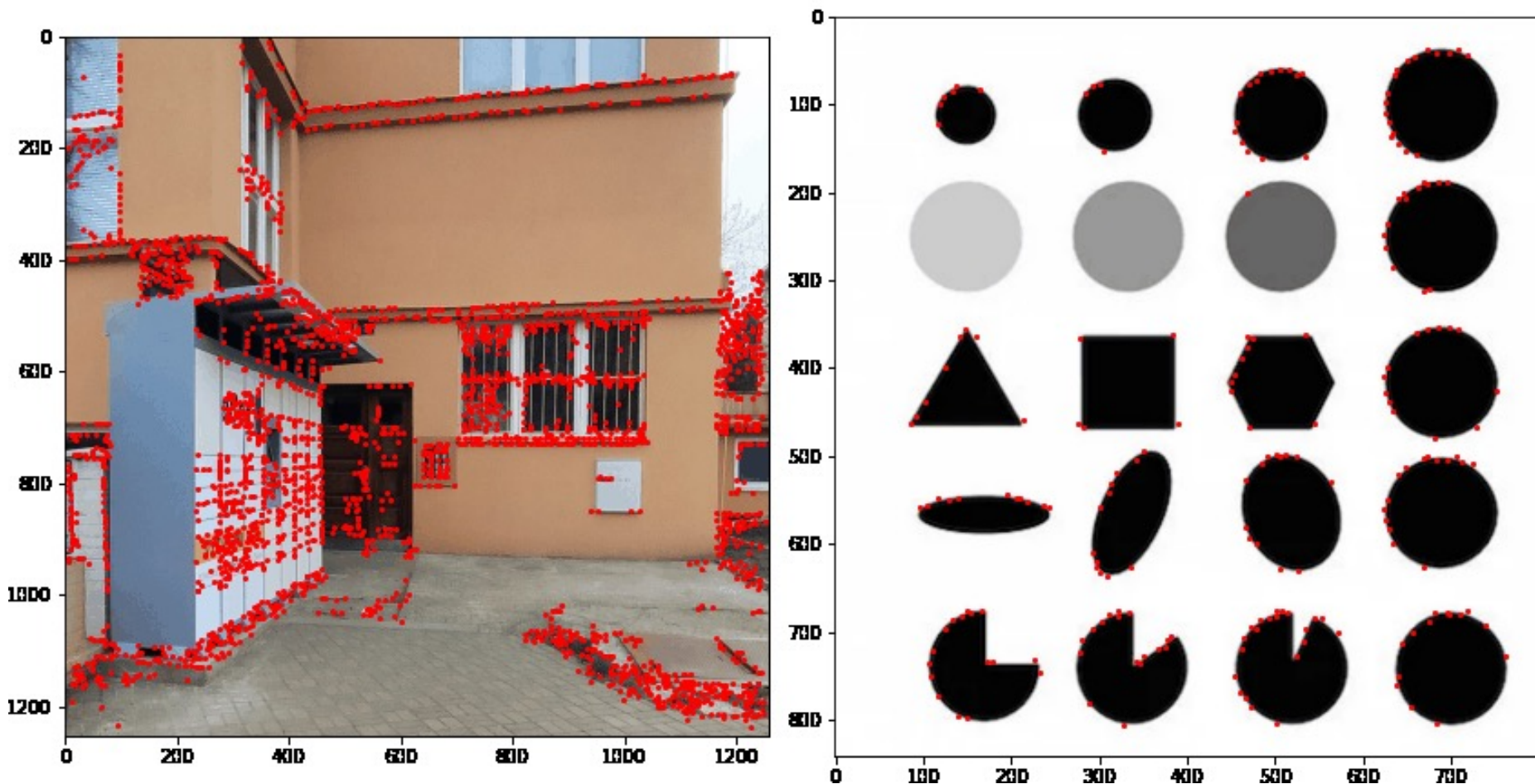
(un-)expected properties of discovered keypoints



Toy model of keypoint detection. Under a constraint of $K = 10$ keypoints and a random spatial distribution of the keypoints, the only consistent way to choose the keypoints is to either only choose white, or only choose black.

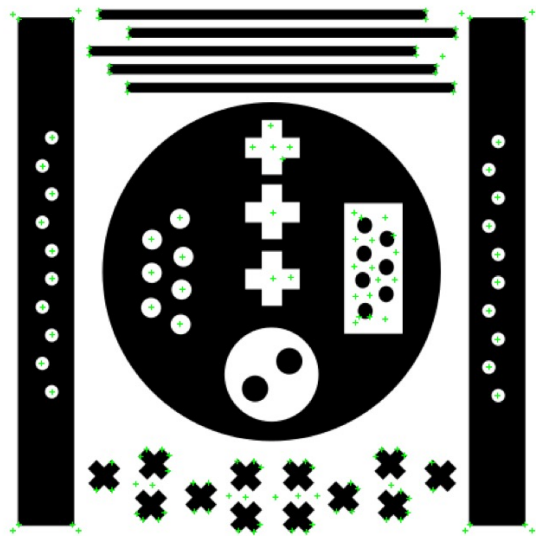
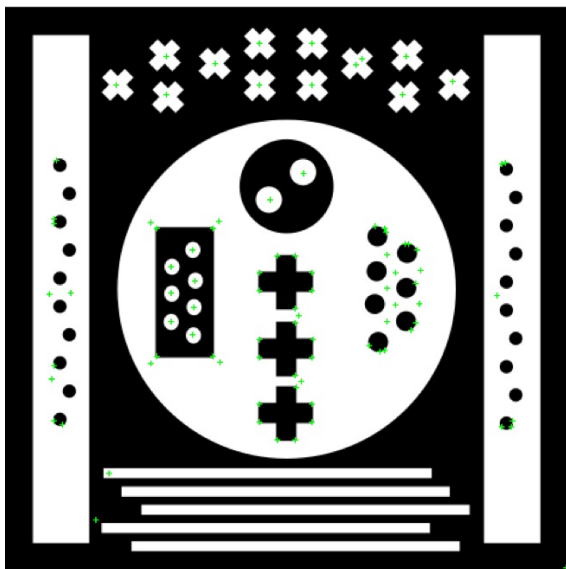
(un-)expected properties of discovered keypoints

KP2D V0 detections



Many discovered keypoint detectors tend to detect dark corners

(un-)expected properties of discovered keypoints



Dark kiki and Light bouba – under different runs, different detectors are discovered by RL

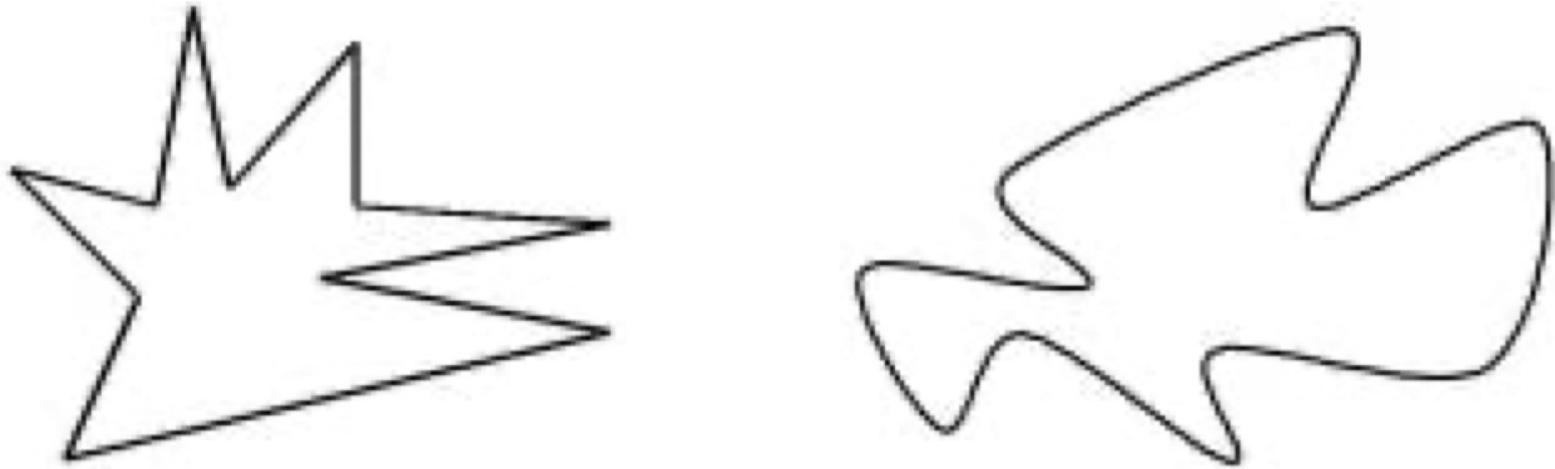


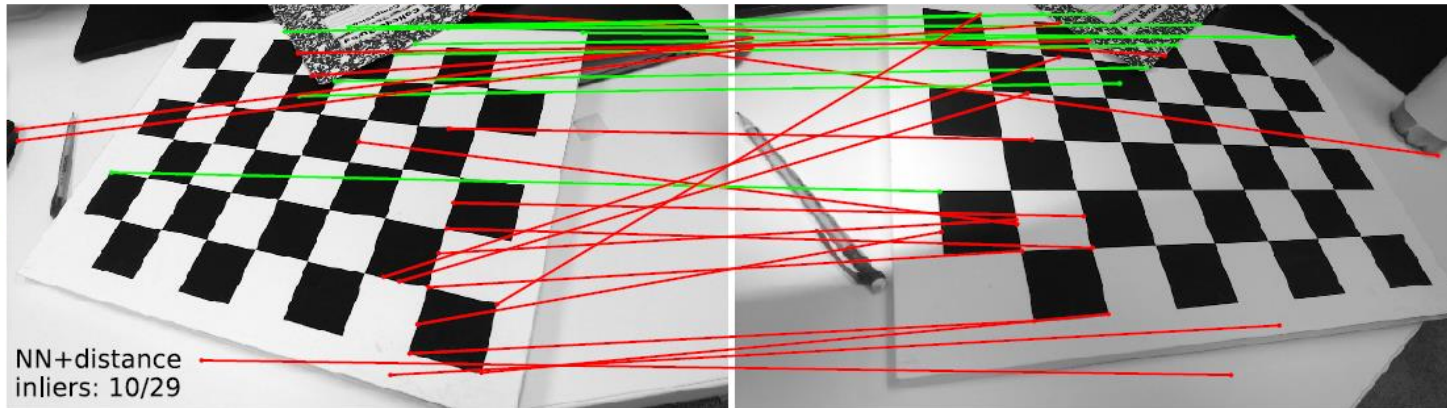
Figure 7. Demonstration of kiki and bouba. Because of the sharp inflection of the visual shape, subjects tend to map the name kiki onto the figure on the left, while the rounded contours of the figure on the right make it more like the rounded auditory inflection of bouba.

**How do you learn
descriptor matcher?
You don't, you learn
better descriptors with
context**

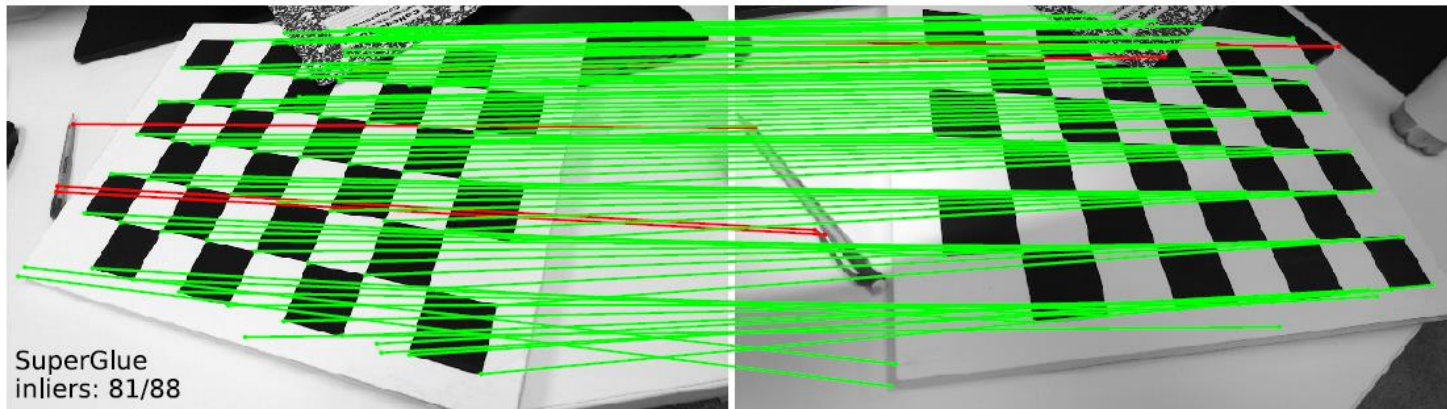
SuperGlue: learned matcher

The importance of context

no
SuperGlue



with
SuperGlue



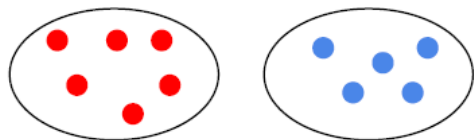
Problem formulation

Inputs



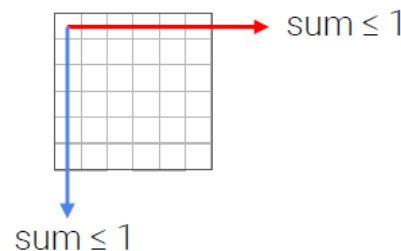
Outputs

- Images **A** and **B**
- **2 sets** of **M**, **N** local features
 - Keypoints: $\mathbf{p}_i := (x, y, c)_i$
 - Coordinates (x, y)
 - Confidence c
 - Visual descriptors: $\mathbf{d}_i$

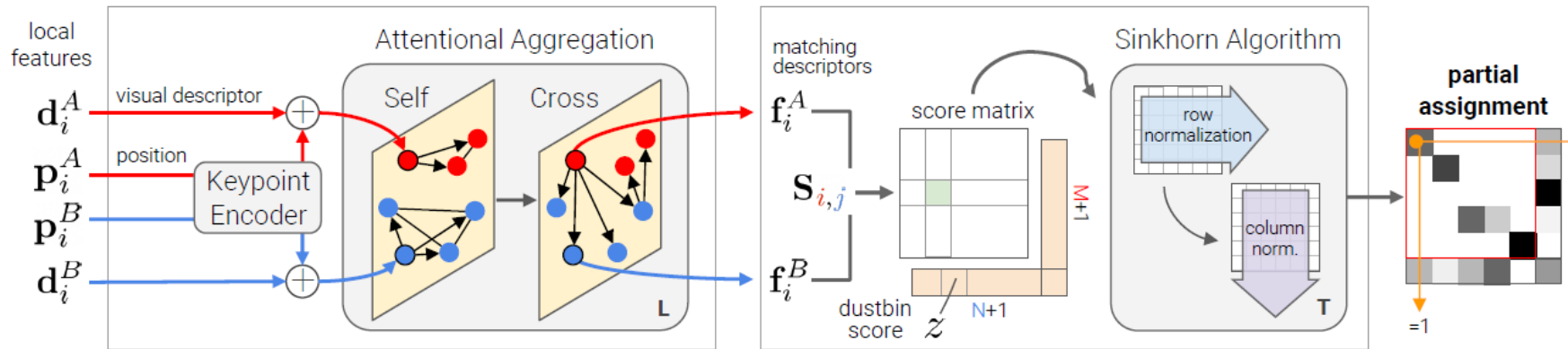


Single a match per keypoint
+ occlusion and noise
→ a **soft partial assignment**:

$$\mathbf{P} \in [0, 1]^{M \times N}$$



SuperGlue: learned matcher



**A Graph Neural Network
with attention**

**Solving a partial
assignment problem**

Encodes **contextual cues** & priors

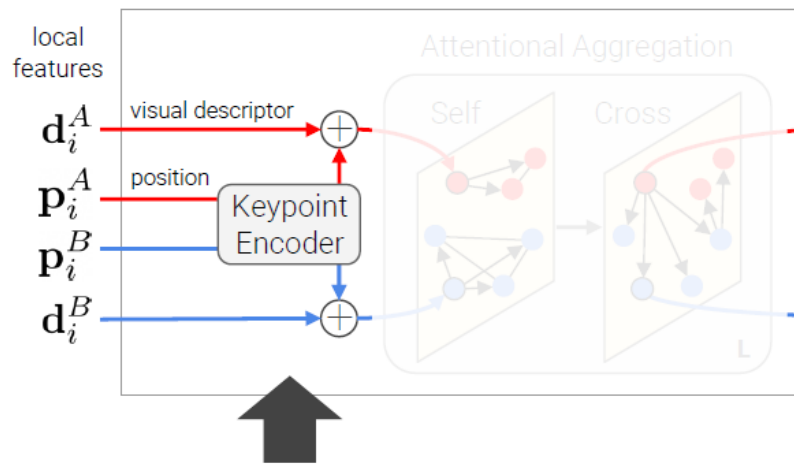
Differentiable **solver**

Reasons about the 3D scene

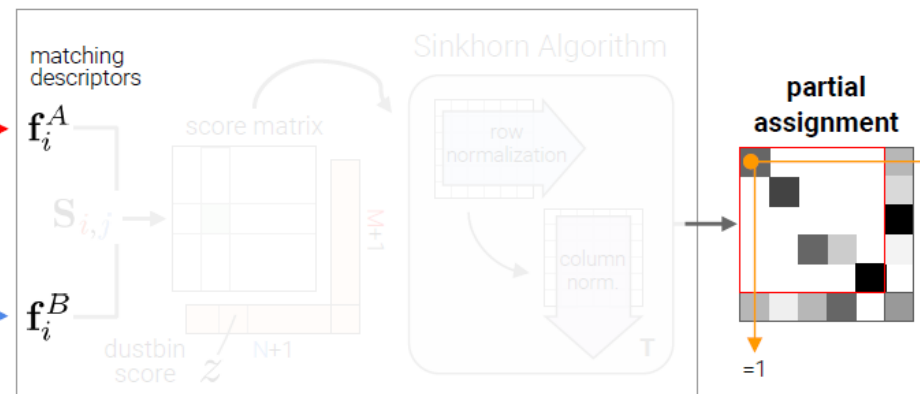
Enforces the assignment constraints
= **domain knowledge**

SuperGlue: learned matcher

Attentional Graph Neural Network



Optimal Matching Layer



- Initial representation for each keypoints i : $^{(0)}\mathbf{x}_i$
- Combines visual appearance and position with an MLP:

$$^{(0)}\mathbf{x}_i = \mathbf{d}_i + \text{MLP}(\mathbf{p}_i)$$

Multi-Layer Perceptron

Attentional Aggregation

- Compute the **message** $\mathbf{m}_{\mathcal{E} \rightarrow i}$ using **self** and **cross attention**
- Soft database retrieval: query $\mathbf{q}_i$, key $\mathbf{k}_j$, and value $\mathbf{v}_j$

$$\mathbf{m}_{\mathcal{E} \rightarrow i} = \sum_{j:(i,j) \in \mathcal{E}} \alpha_{ij} \mathbf{v}_j \quad \left| \quad \begin{aligned} \mathbf{q}_i &= \mathbf{W}_1^{(\ell)} \mathbf{x}_i + \mathbf{b}_1 \\ \begin{bmatrix} \mathbf{k}_j \\ \mathbf{v}_j \end{bmatrix} &= \begin{bmatrix} \mathbf{W}_2 \\ \mathbf{W}_3 \end{bmatrix}^{(\ell)} \mathbf{x}_j + \begin{bmatrix} \mathbf{b}_2 \\ \mathbf{b}_3 \end{bmatrix} \end{aligned}$$
$$\alpha_{ij} = \text{Softmax}_j (\mathbf{q}_i^\top \mathbf{k}_j)$$

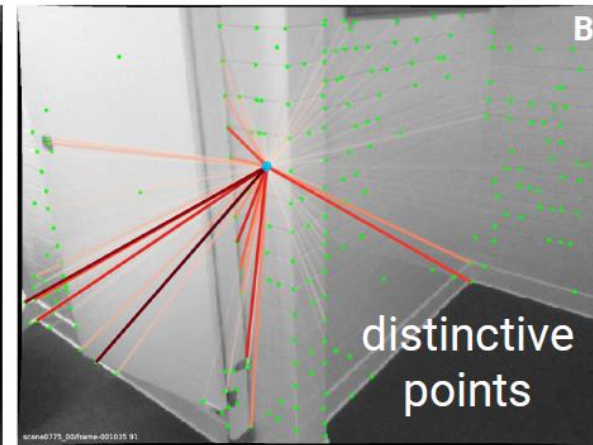
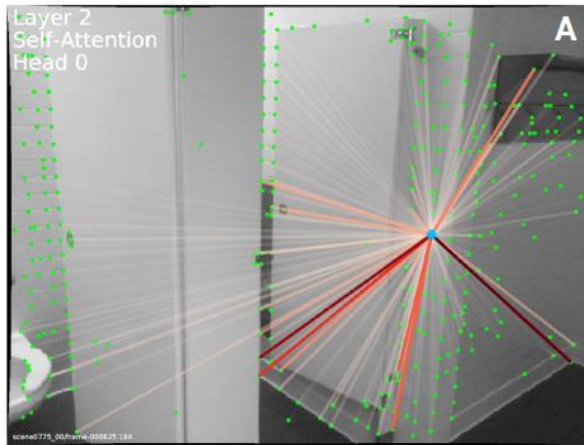


[Vaswani et al, 2017]

SuperGlue: learned matcher

Self-attention

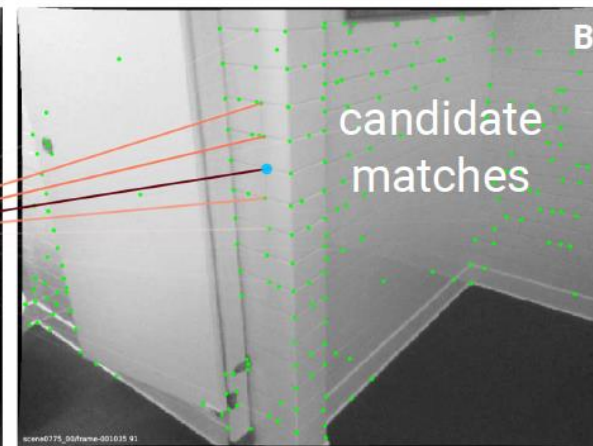
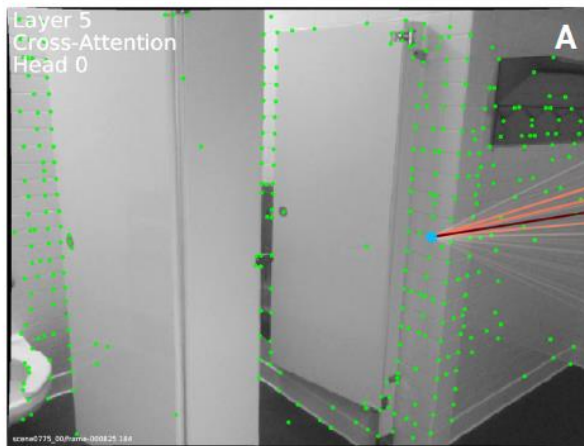
= intra-image
information
flow



Cross-attention

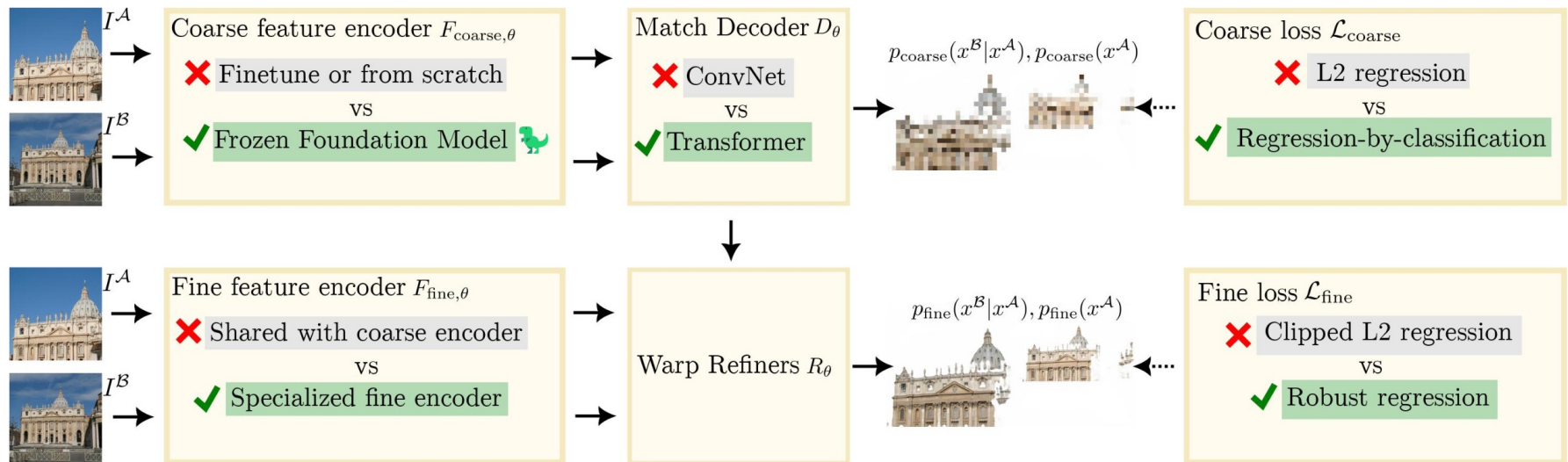
= inter-image

Attention builds a
**soft, dynamic,
sparse graph**



Ideas:

- Match coarse to fine, the coarse is similar to SuperGlue on dense DINOv2 features
- Use fine-resolution backbone (VGG19) + regressor head to “upscale” -- find pixel-wise warp similar to optical flow

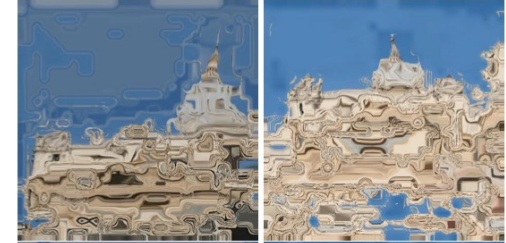


- Using pretrained features can give good start for matching model.

Images



VGG19



ResNet50



DINOv2



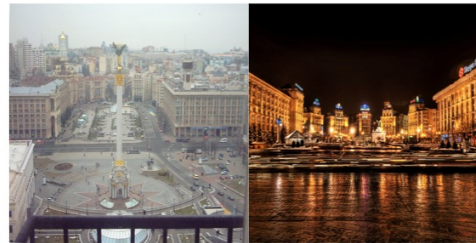
RoMa



RoMa solves WxBS dataset



Method	mAA@ $\rightarrow$	10px $\uparrow$
DISK [53] NeurIps'20		35.5
DISK + LightGlue [31, 53] ICCV'23		41.7
SuperPoint + SuperGlue [14, 41] CVPR'20		31.4
LoFTR [44] CVPR'21		55.4
DKM [17] CVPR'23		58.9
RoMa		80.1



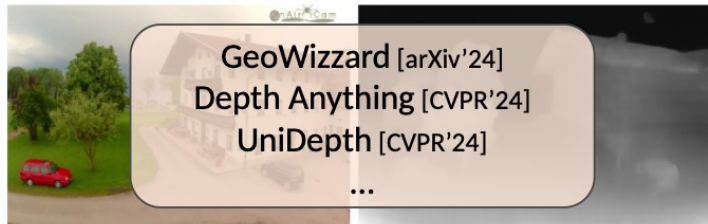
Directly to 3D

*3R family

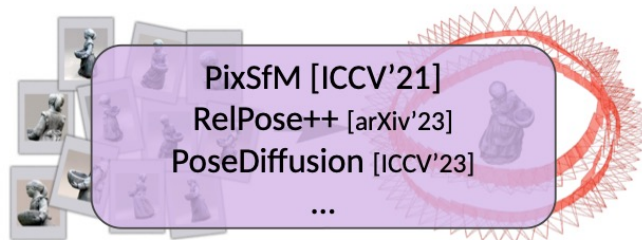
[Slides source](#)

What is 3D vision?

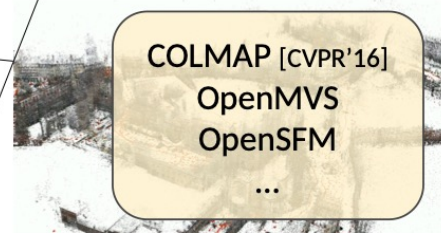
Monocular Depth estimation



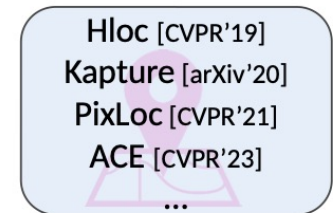
Point matching



Multi-view pose estimation



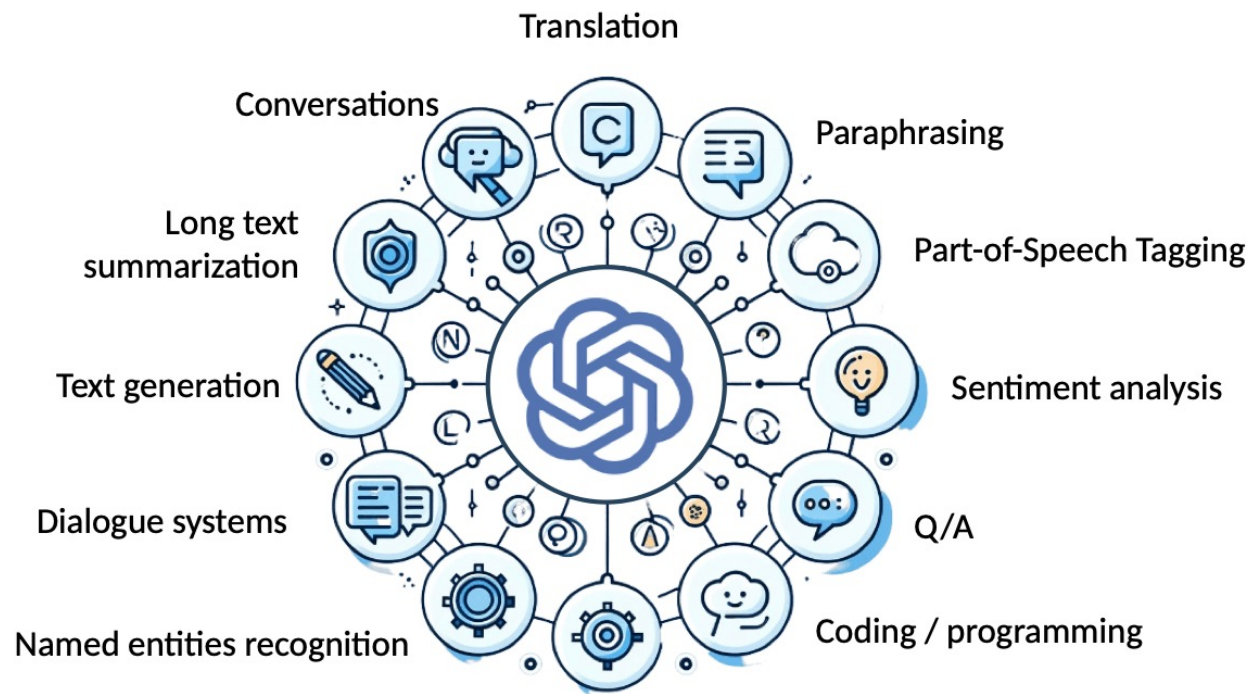
Large-scale 3D reconstruction



Visual Localization

... and many more: SLAM, calibration, MVS, ...

Why seek a unified model? The case of NLP



CroCo: Self-supervised learning with Cross-View Completion

[NeurIPS'22] [ICCV'23]

A guessing game:
what's behind the mask?



CroCo: Self-supervised learning with Cross-View Completion

[NeurIPS'22] [ICCV'23]



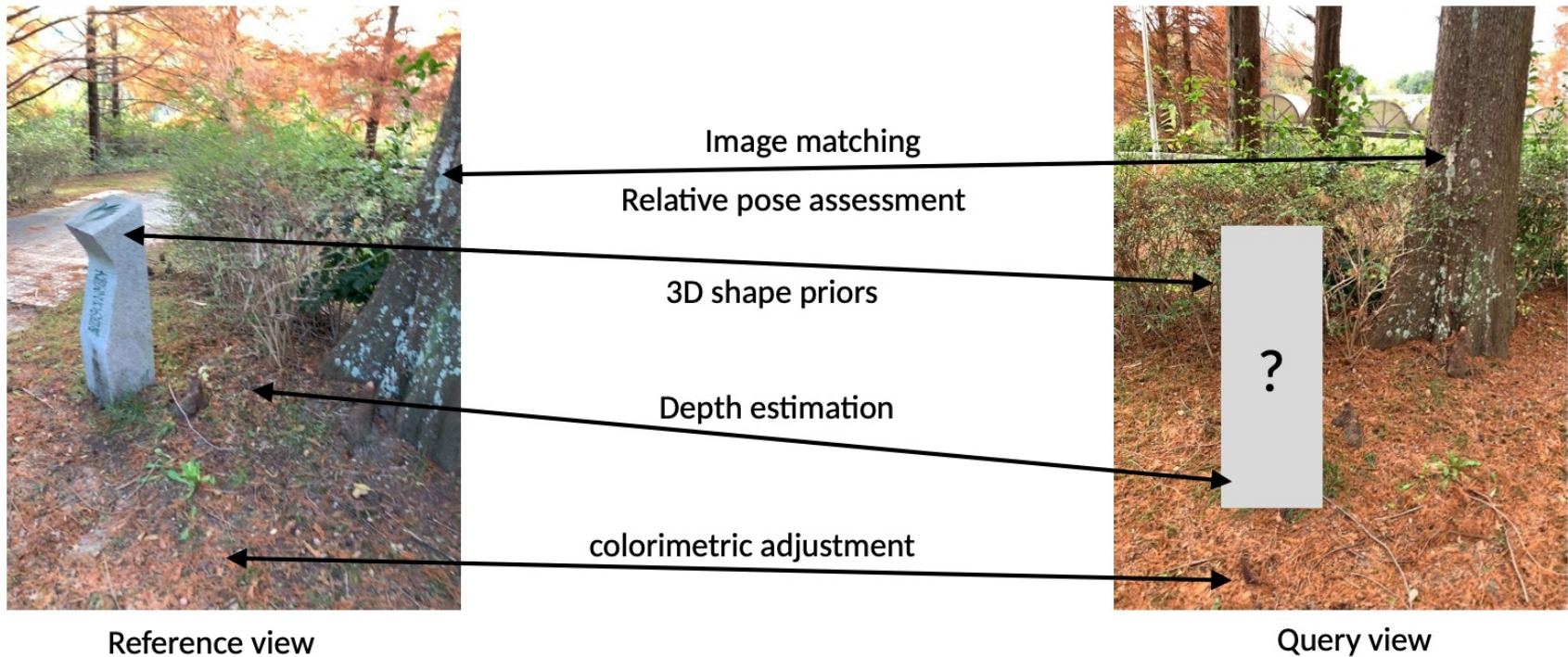
Reference view



Query view

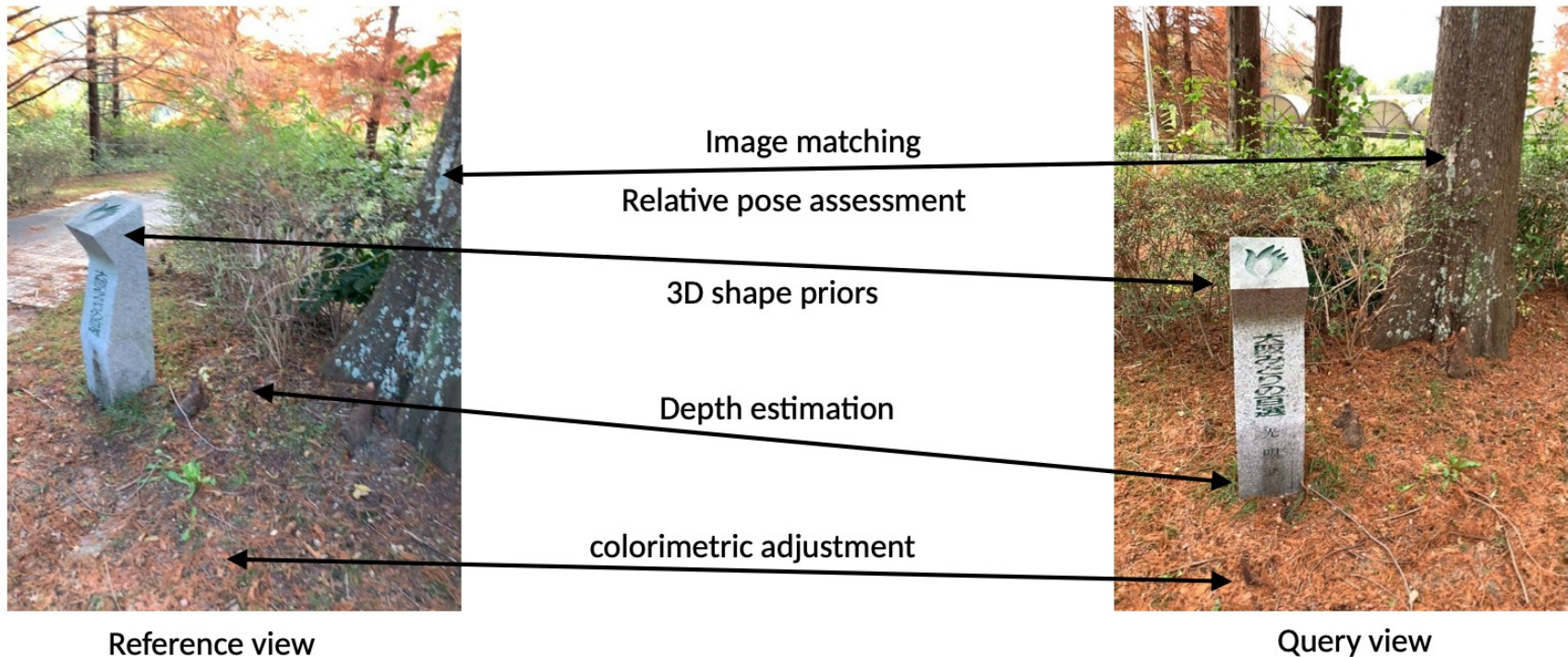
CroCo: Self-supervised learning with Cross-View Completion

[NeurIPS'22] [ICCV'23]

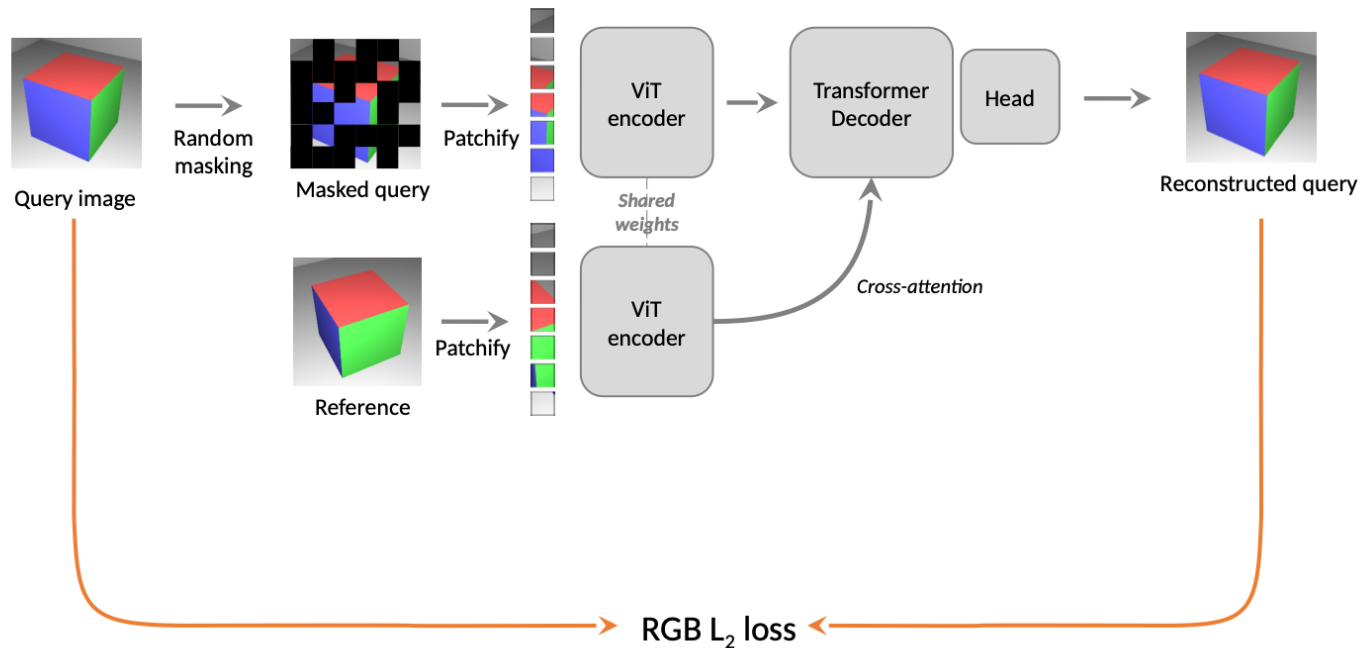


CroCo: Self-supervised learning with Cross-View Completion

[NeurIPS'22] [ICCV'23]



CroCo: Self-supervised learning with Cross-View Completion



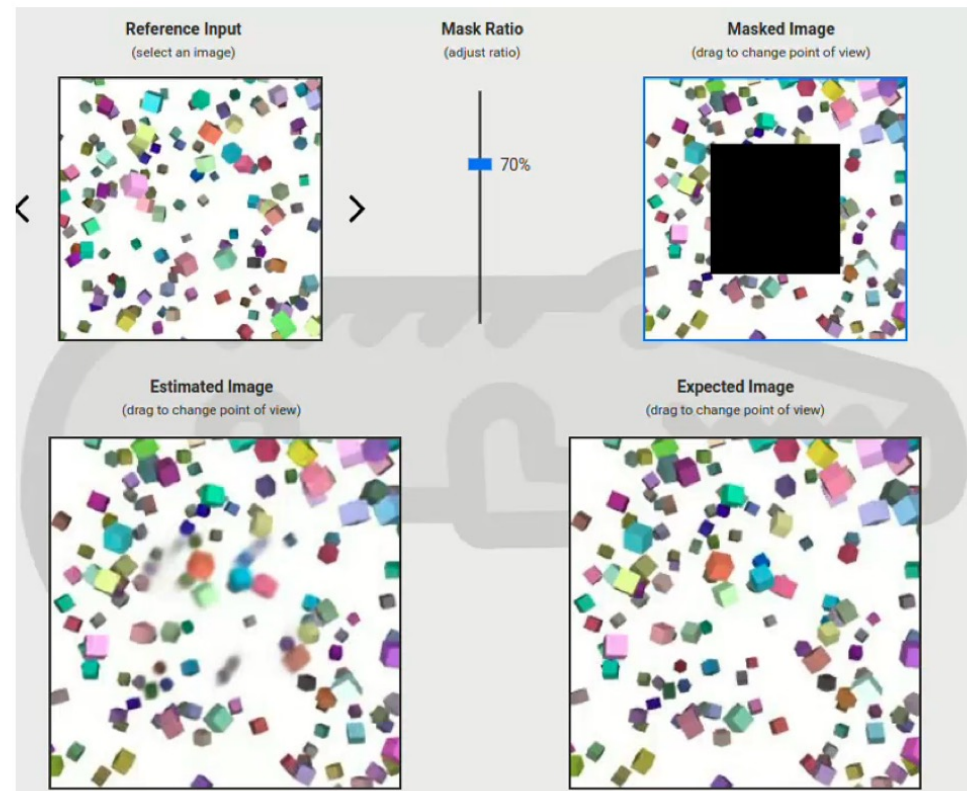
CroCo: Self-supervised learning with Cross-View Completion

Proof of concept:

- training with synthetic random scenes
- Test scene never seen before!

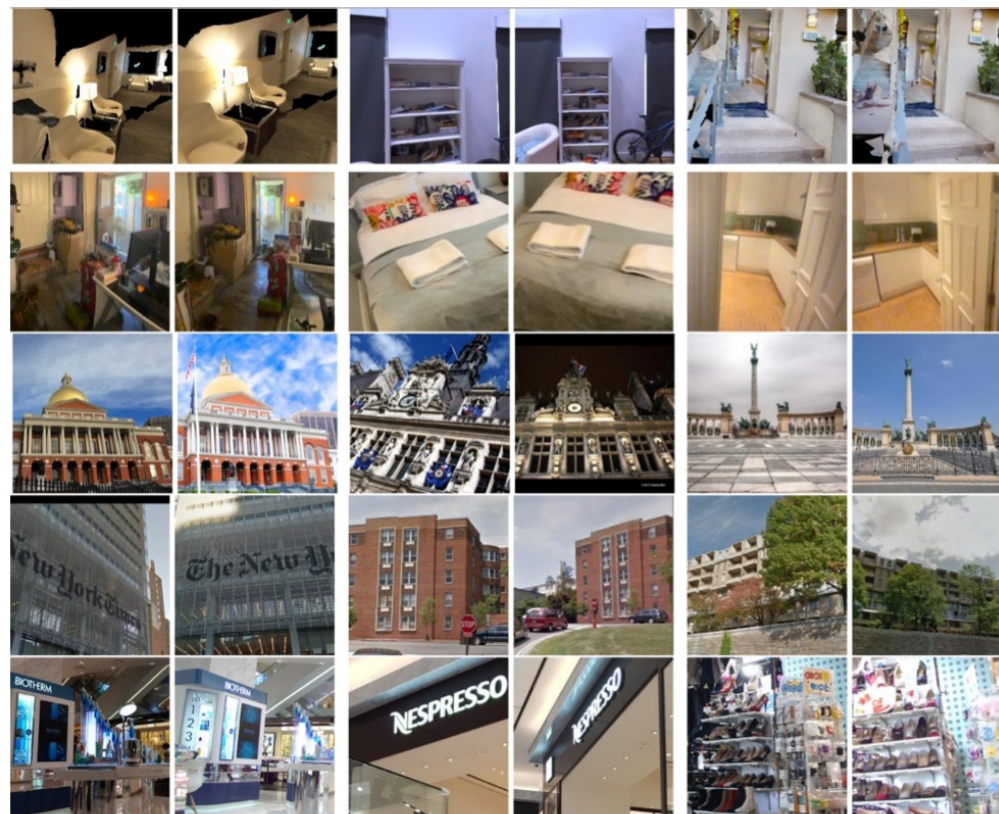
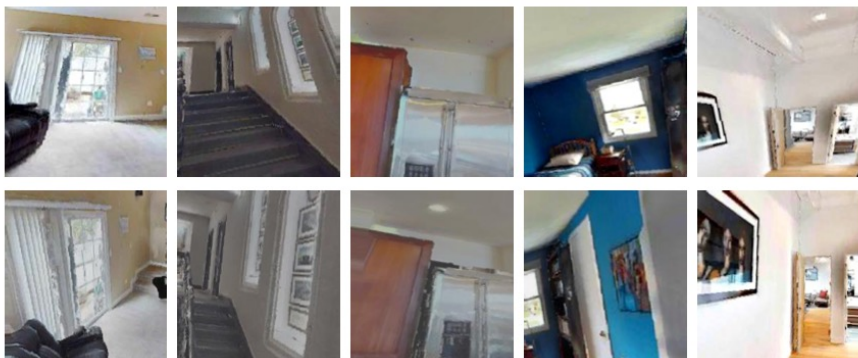
What solving this implies:

- Match the query and reference images
- Estimate the relative pose
- Infer an object-centric 3D reconstruction of the reference scene
- Align (rotate) the reference scene in 3D
- Render the reference scene based on imagined



Pre-training data

2M image pairs from the Habitat simulator
[Savva *et al.*, ICCV'19]



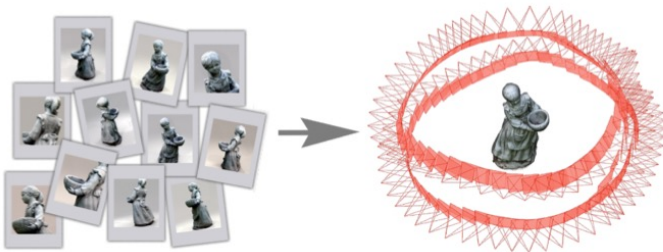
+ 5M training real image pairs

What is 3D vision?

Monocular Depth estimation



Point matching



Multi-view pose estimation



**Dense 3D
reconstruction**



Visual
Localization

... and many more: SLAM, calibration, MVS, ...

Unifying all 3D vision tasks?

COLMAP's official restrictions

Capture images with good texture.

Avoid texture-less images

Capture images at similar illumination conditions

Avoid high dynamic range scenes

Avoid specularities on shiny surfaces

Capture images with high visual overlap.

each object in at least 3 images – the more the better

Capture images from different viewpoints.

Do not take images from the same location by only rotating the camera, e.g., make a few steps after each shot

At the same time, try to have enough images from a relatively similar viewpoint

“Structure-from-Motion Revisited”, “Pixelwise View Selection for Unstructured Multi-View Stereo”, Schonberger et al., in CVPR’16 & ECCV’16

Unifying all 3D vision tasks?

3D reconstruction is a “super-task”

- intrinsically connected to all other 3DV tasks

Current solution is problematic

- Brittle, requires enough *images & overlap & textures & viewpoints*
- Heavily handcrafted at all levels
 - An engineering hell!
- Multiple minimal problems solved sequentially
 - No internal collaboration between them
- Slow

Unifying all 3D vision tasks?

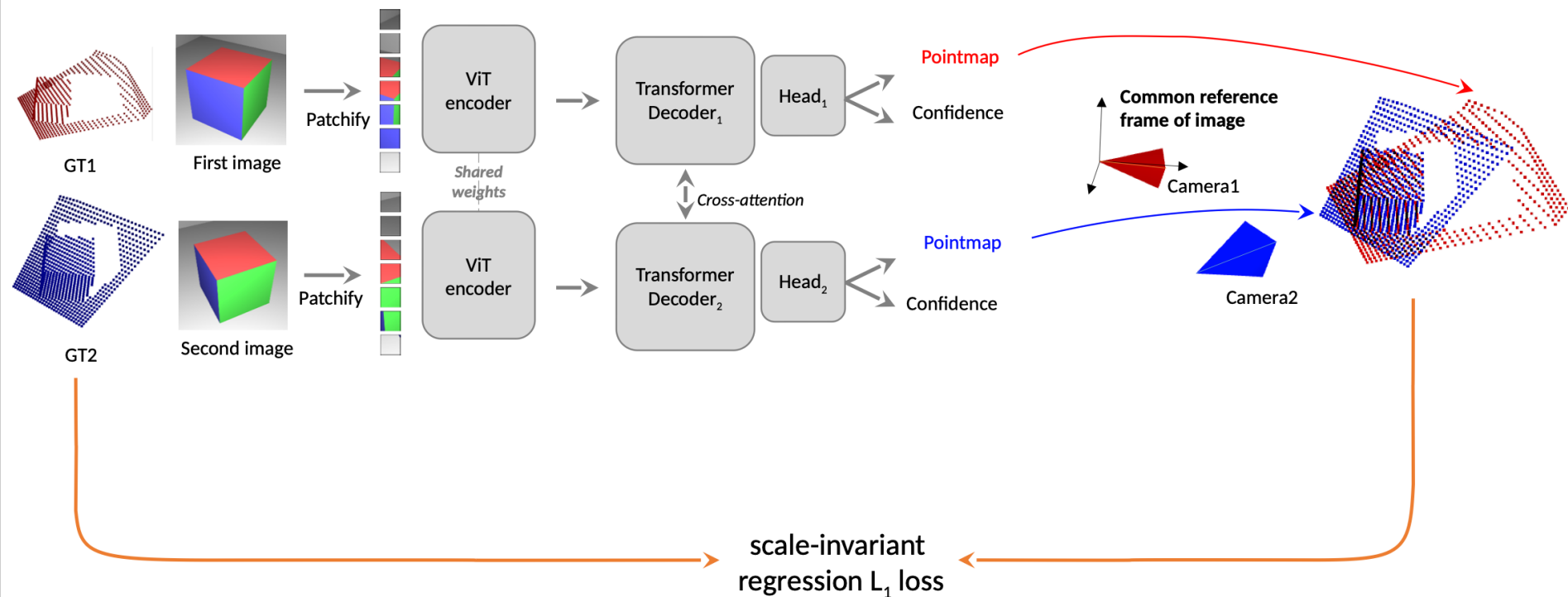
3D reconstruction is a “super-task”

- intrinsically connected to all other 3DV tasks

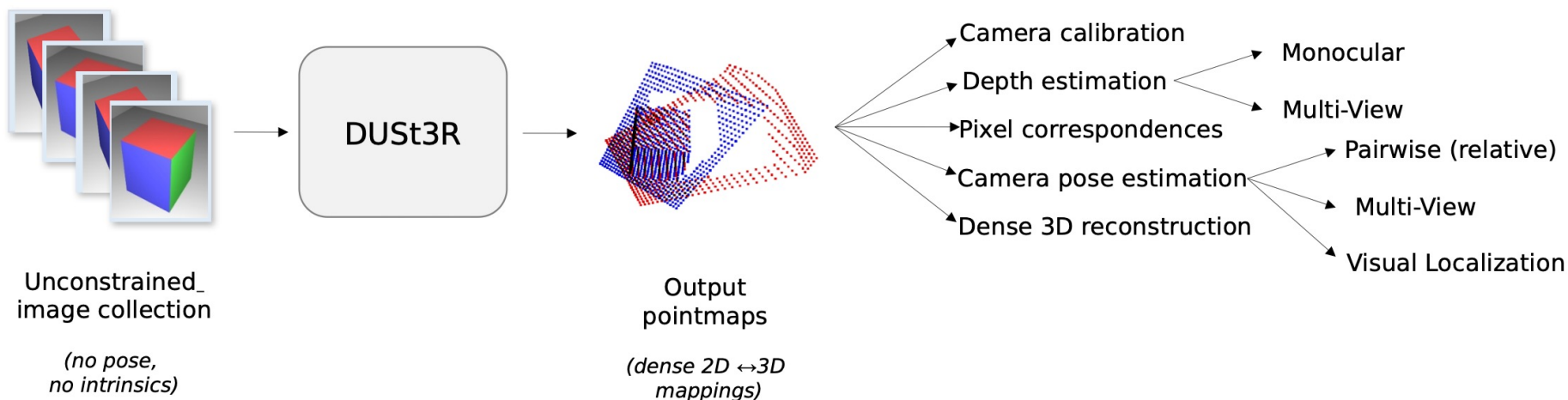
Current solution is problematic

- Brittle, requires enough *images & overlap & textures & viewpoints*
- Heavily handcrafted at all levels
 - An engineering hell!
- Multiple minimal problems solved sequentially
 - No internal collaboration between them
- Slow

DUS_t3R: Dense Unconstrained Stereo 3D Reconstruction



DUS_t3R: Dense Unconstrained Stereo 3D Reconstruction



DUS_t3R: Dense Unconstrained Stereo 3D Reconstruction

Training data

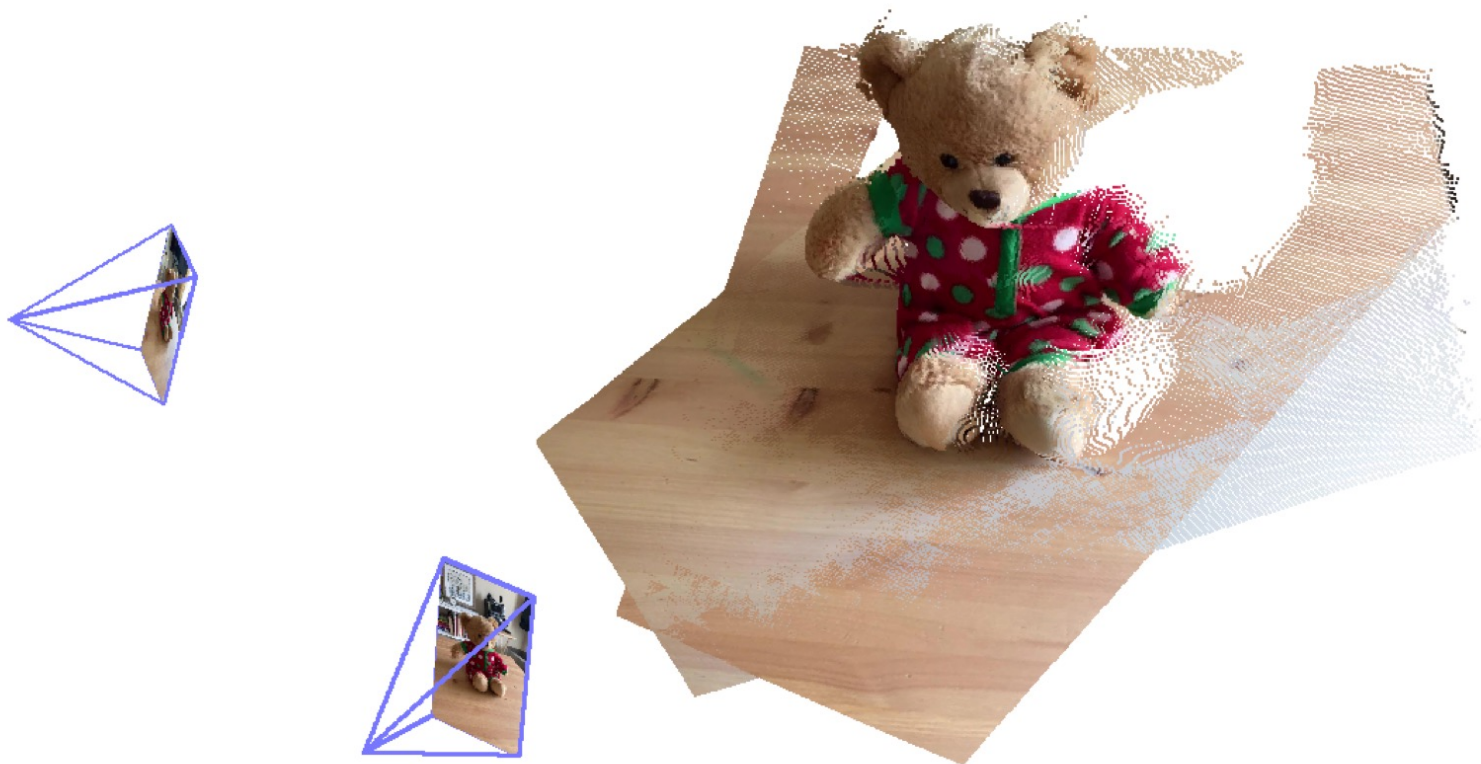
Datasets	Type	N Pairs
Habitat [103]	Indoor / Synthetic	1000k
CO3Dv2 [93]	Object-centric	941k
ScanNet++ [165]	Indoor / Real	224k
ArkitScenes [25]	Indoor / Real	2040k
Static Thing 3D [68]	Object / Synthetic	337k
MegaDepth [55]	Outdoor / Real	1761k
BlendedMVS [161]	Outdoor / Synthetic	1062k
Waymo [121]	Outdoor / Real	1100k



Jointly recovering cameras and scene

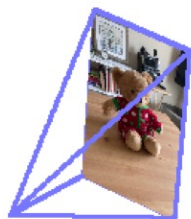


Jointly recovering cameras and scene



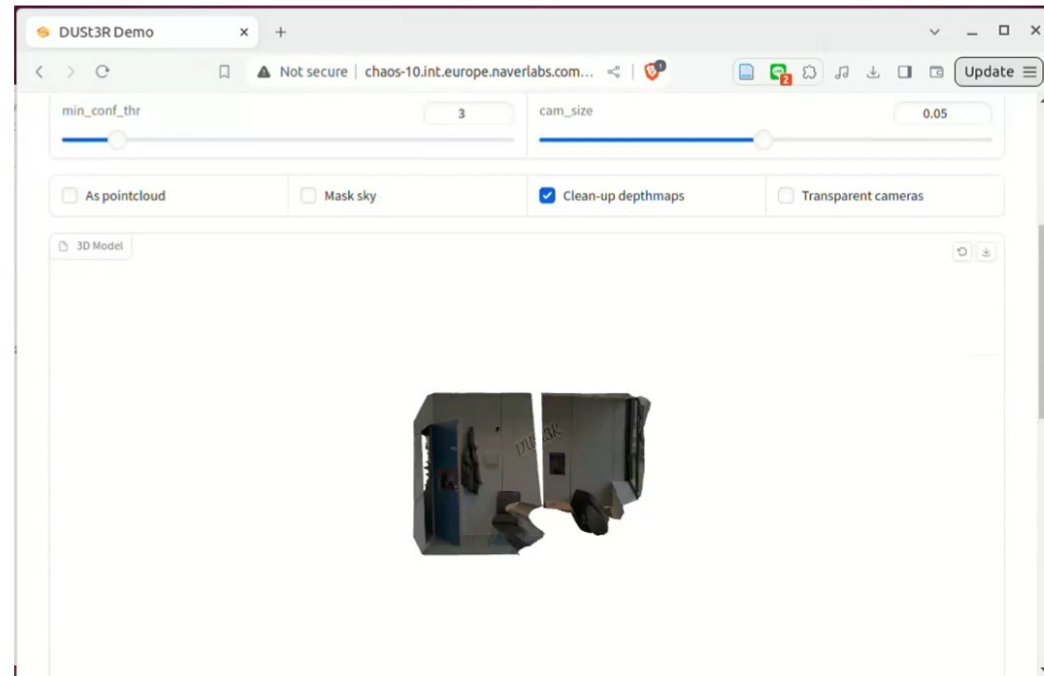
Monocular Input

Feed same image twice



DUST3R

“impossible matching” = 3D reconstruction without any overlap!



DUS_t3R: limitations

Not all routes leads to accurate visual localization

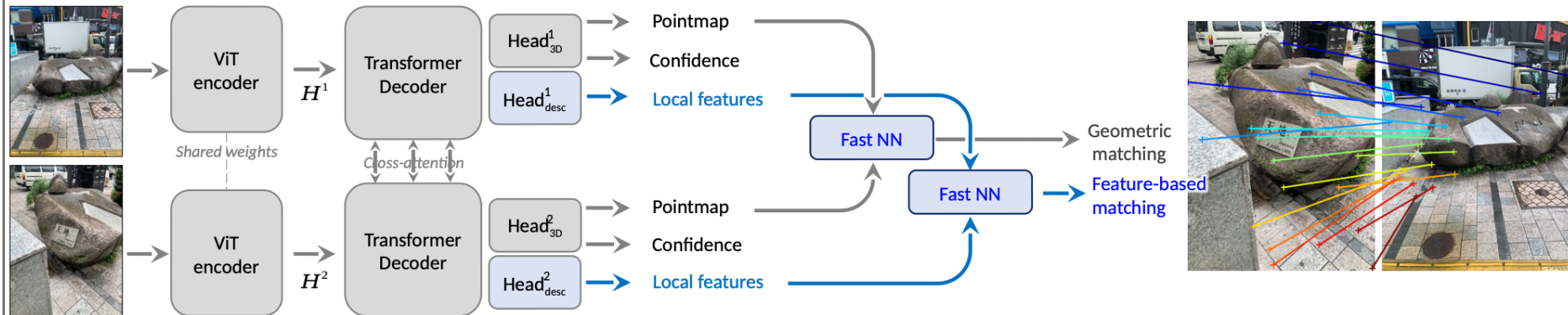
- Route 1: DUS_t3R → NN in 3D space → pixel correspondences → PnP
- Route 2: DUS_t3R → PnP

Methods	GT	7Scenes (Indoor) [48]						
	Focals	Chess	Fire	Heads	Office	Pumpkin	Kitchen	Stairs
DUS _t 3R 512 from 2D-matching	✓	3/0.97	3/0.95	2/1.37	3/1.01	4/1.14	4/1.34	11/2.84
DUS _t 3R 512 from scaled rel-pose	×	5/1.08	5/1.18	4/1.33	6/1.05	7/1.25	6/1.37	26/3.56

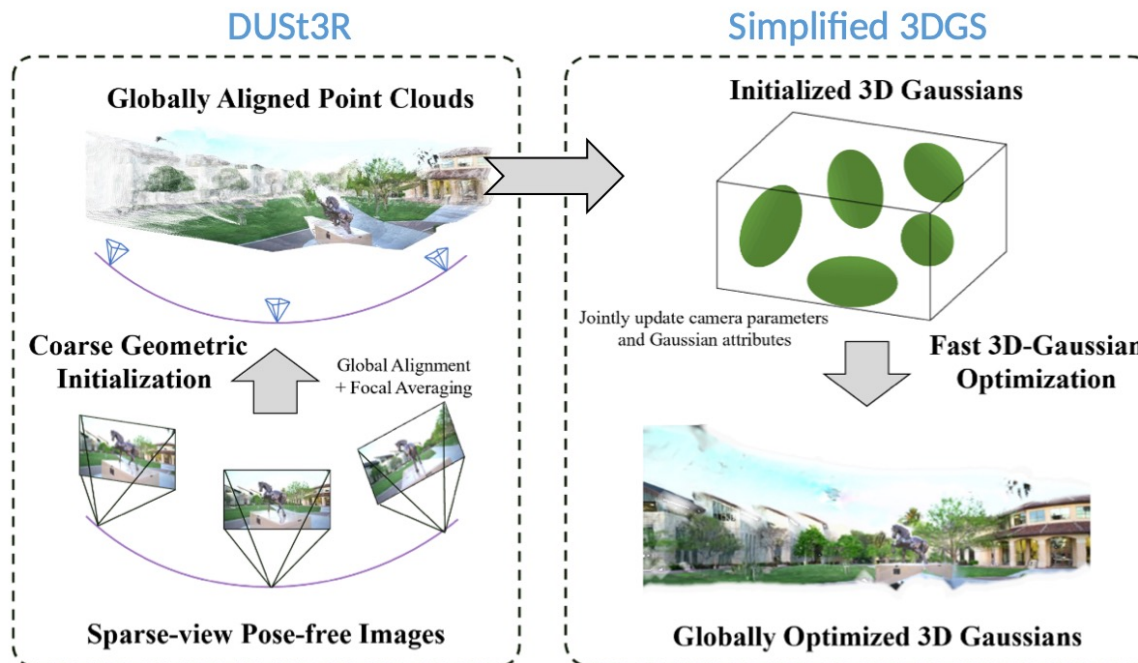
Best results obtained from pixel correspondences

- but DUS_t3R is not trained explicitly for matching
- What if we did?

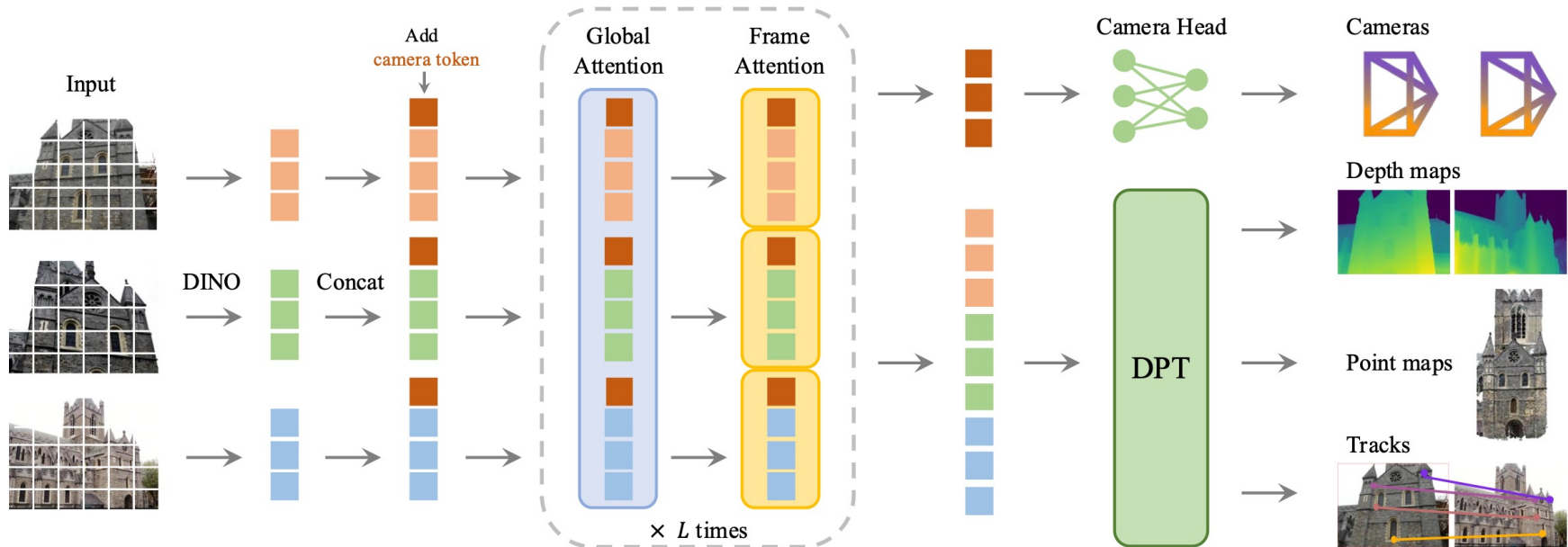
MASt3R: Matching And Stereo 3D Reconstruction



InstantSplat: Novel View Rendering *from scratch* in seconds



InstantSplat: Unbounded Sparse-view Pose-free Gaussian Splatting in 40 Seconds. Zhiwen Fan et. al., [arXiv'24]. <https://instantsplat.github.io>



VGGT starts from DINOv2 tokens and jointly processes multiple images.
It predicts: pointmap, depth, camera pose, tracks.

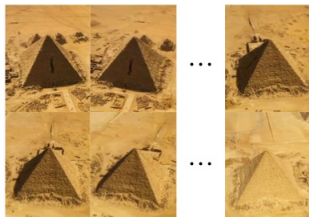
Input



Single view

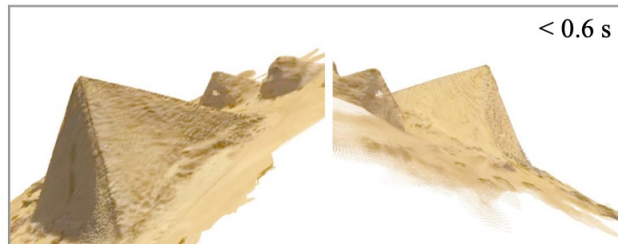
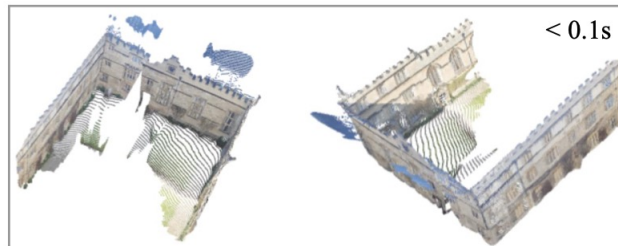


Two views

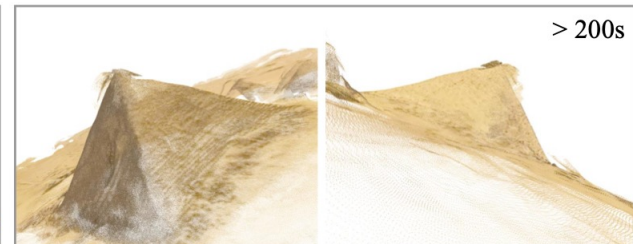


32 views

VGGT



DUS_t3R



Bonus: how to evaluate

Image Matching Challenge